

Bitácora de Desarrollo — Proyecto PokéTienda Pearl

Asignatura: DSY1104 — Desarrollo Web Frontend

Proyecto: Tienda online temática Pokémon — Edición Perla

Documento: Bitácora de intervención sobre `carrito.html`

Versión del documento: 1.1

Fecha: 2025

Responsable técnico: Desarrollador Fullstack contratado

Cliente: Equipo de desarrollo DSY1104 (Gabriel Flores, Daniel Rangel, Hernaldo Silva)

1. Objetivo de la bitácora

Dejar constancia formal de la intervención realizada sobre el archivo `carrito.html` tras la modificación efectuada por el desarrollador CSS externo, documentando el diagnóstico, los cambios aplicados, las decisiones de diseño y su alineación con la identidad visual v2.0 y los requerimientos del ERS.

Esta bitácora complementa la versión 1.0 del documento y sirve como:

- Evidencia de continuidad del trabajo sobre el módulo público.
 - Registro del estado actual del archivo `carrito.html` previo a la implementación de la capa JavaScript.
 - Referencia para el equipo de desarrollo en futuras iteraciones.
-

2. Antecedentes y contexto de la intervención

2.1 Situación previa

En la iteración anterior se entregó una versión funcional y semánticamente completa de `carrito.html`, alineada con el **RF-04** del ERS, el mockup de la **Figura 15** y la identidad visual v2.0.

Posteriormente, el cliente incorporó a un **desarrollador CSS externo** para acelerar la implementación de la capa de presentación. Durante ese trabajo, el archivo `carrito.html` fue **refactorizado en su estructura** para facilitar el estilizado.

2.2 Resultado de la refactorización del CSS dev

El desarrollador CSS introdujo una nueva nomenclatura y estructura de componentes, que el cliente valoró positivamente:

Componente nuevo	Descripción
<code>.carrito-view</code>	Contenedor principal de la vista del carrito
<code>.carrito-view__layout</code>	Grilla de dos columnas (lista + resumen)
<code>.carrito-items</code>	Contenedor de la lista de ítems
<code>.carrito-item</code>	Tarjeta individual de producto en el carrito
<code>.carrito-item__info</code>	Bloque de información textual del ítem
<code>.carrito-item__acciones</code>	Bloque de controles (precio, cantidad, quitar)
<code>.selector-cantidad</code>	Componente nuevo de control de cantidad con + / -
<code>.carrito-resumen</code>	Aside con el resumen de compra
<code>.carrito-resumen__fila</code>	Filas de subtotal, envío, etc.
<code>.carrito-resumen__total</code>	Bloque destacado del TOTAL

Sin embargo, la refactorización dejó el **contenido de los ítems en estado de marcador de posición (placeholder)**: `src="..."`, `alt="..."`, "Producto #1", "Descripción breve...", precios genéricos. Esto restaba credibilidad al carrito de cara a la presentación y a la evaluación del proyecto.

2.3 Solicitud del cliente

"Hay ciertas características que le gustaron, pero la información dentro del carrito terminó por ser poco representativa. Revisa el archivo y haz mejoras para que el contenido respecto al producto en el carrito sea más representativo. Mantén el estilo y la coherencia con todo tu trabajo anterior."

3. Diagnóstico

Tras la revisión del archivo enviado por el cliente, se identificaron las siguientes deficiencias:

#	Deficiencia	Impacto
1	<code>src=""</code> y <code>alt=""</code> en las imágenes	Los ítems no mostraban imagen real; se veía el ícono de imagen rota
2	Nombres genéricos ("Producto #1")	No permitía identificar el producto
3	Descripción genérica ("Descripción breve...")	Nulo valor informativo
4	Sin SKU ni estado de stock	No aportaba trazabilidad ni contexto de disponibilidad
5	Precio único sin distinguir unitario vs. subtotal	Ambigüedad en el cálculo mostrado
6	Sin categoría del producto	Pérdida de contexto de navegación
7	Sin enlaces a <code>detalle-producto.html</code>	Flujo de usuario incompleto
8	Selector de cantidad sin <code>data-*</code>	Imposible conectarlo con <code>carrito.js</code>
9	Falta de subtítulo y contador total de productos	El usuario no sabía cuántos ítems tenía

4. Cambios aplicados

4.1 Criterios rectores

La intervención respetó tres principios:

1. **Preservar** íntegramente las clases, IDs y estructura introducidas por el desarrollador CSS, para no romper su trabajo de estilizado.
2. **Enriquecer** el contenido de cada ítem con información real, verificable y coherente con el catálogo ya definido en `productos.html` y `detalle-producto.html`.
3. **Mantener** la coherencia con la identidad visual v2.0 y la semántica HTML5 exigida por la rúbrica.

4.2 Detalle de cambios por componente

4.2.1 Lista de ítems (`.carrito-items`)

Se reemplazaron los ítems de ejemplo por **tres productos reales del catálogo**:

Ítem	Producto	Categoría	SKU	Precio unitario
1	Figura Palkia Legendaria	Figuras · Edición Perla	P001	\$24.990
2	Set de Cartas Sinnoh	Cartas · TCG	P003	\$8.990
3	Estuche Poké Ball Pearl	Accesorios · Colección	P006	\$7.990

Subtotal calculado inicial: \$41.970 CLP.

4.2.2 Enriquecimiento de cada ítem

Cada `.carrito-item` fue dotado con la siguiente información representativa:

- **Imagen real** con `src` y `alt` descriptivo, envuelta en un enlace a su ficha de detalle.
- **Categoría** con modificador de subcategoría (ej. "Figuras · Edición Perla").
- **Nombre del producto** como enlace a `detalle-producto.html?id=XXX`.
- **Descripción breve** con datos concretos: materiales, dimensiones, contenido del empaque.
- **Lista de detalles** con:
 - `SKU` en etiqueta `<code>`.
 - Estado de stock con `badge--exito` o `badge--advertencia`.
- **Precio unitario** con indicador `c/u`.
- **Subtotal por línea** destacado con la clase `.precio` (tipografía Fredoka One, color amarillo `#FFC107`).
- **Selector de cantidad** con `data-accion="restar|sumar"` y `data-id` para event delegation.
- **Botón "Quitar"** con ícono SVG y `data-accion="eliminar"`, `aria-label` descriptivo.

4.2.3 Cabecera de la vista

Se añadió un bloque `<header class="carrito-view__cabecera">` con:

- `<h1>` "Mi carrito de compras" (antes solo título suelto).
- Subtítulo dinámico: *"Tienes 3 productos en tu carrito"* con `id="carrito-cantidad-total"` para actualización vía JS.
- El H1 ahora está asociado a la sección con `aria-labelledby`, cumpliendo la rúbrica semántica.

4.2.4 Estado vacío

Se introdujo un bloque `.carrito-vacio` (`hidden` por defecto) con:

- Ícono SVG del carrito.
- Título y texto informativos.
- CTA "Explorar productos".

`carrito.js` mostrará este bloque y ocultará `#carrito-items` cuando el carrito esté vacío.

4.2.5 Resumen lateral (`.carrito-resumen`)

Se conservó íntegramente la estructura del CSS dev, enriqueciendo el contenido con:

- Desglose semántico con `<dl>` / `<dt>` / `<dd>`.
- Fila de descuento oculta (`hidden`) hasta que se aplique un cupón.
- Bloque de cupón aplicado (`#cupon-aplicado`) con botón de quitar.
- Nota de envío gratis.
- Botón Pagar con clase `boton--exito` (verde `#28A745`, según guía de aplicación).
- Lista de sellos de confianza.
- Feedback general con `role="status"` y `aria-live="polite"`.

4.2.6 Datos estructurados para JavaScript

Todos los ítems y controles fueron dotados de atributos `data-*` consistentes:

Atributo	Ubicación	Propósito
<code>data-id</code>	<code><article></code> , controles, subtotal	Identificar producto
<code>data-precio</code>	<code><article></code>	Precio unitario base
<code>data-nombre</code>	<code><article></code>	Nombre para el carrito
<code>data-accion="restar sumar eliminar agregar"</code>	Botones	Enrutar event delegation

5. Coherencia con el trabajo anterior

5.1 Header y footer

Se mantuvieron **idénticos** a las otras 15 vistas del proyecto: mismos IDs (`#carrito`, `#carrito-contador`, `#nav-principal`, `#nav-toggle`), mismas clases, misma estructura. Esto garantiza que `carrito.js` y `main.js` sigan funcionando sin modificaciones.

Se añadió `aria-label="Carrito de compras: 3 productos"` y `aria-current="page"` al enlace del carrito del header, para reflejar el contexto actual.

5.2 Identidad visual v2.0

- Colores y clases coherentes con la guía: `precio`, `badge--exito`, `badge--advertencia`, `boton--primario`, `boton--secundario`, `boton--exito`.
- Tipografía respetada: `precio` usa Fredoka One + `#FFC107`; títulos en Orbitron; cuerpo en Inter.
- No se introdujo ninguna paleta ni tipografía nueva.

5.3 Requerimiento RF-04

Sub-requerimiento	Estado
Listado de productos con imagen, nombre, precio y control de cantidad	✓
Cálculo automático del total	✓ Estructura lista (IDs <code>#carrito-subtotal</code> , <code>#carrito-total</code> , etc.)
Campo para ingreso de cupón	✓ <code>#form-cupon</code> , <code>#cupon-input</code> , <code>#btn-aplicar-cupon</code>
Botón "Pagar"	✓ <code>#btn-pagar</code>
Persistencia en LocalStorage	🕒 Delegado a <code>carrito.js</code>

6. Próximos pasos

6.1 Implementación de `carrito.js`

Con la estructura HTML ya consolidada, el siguiente entregable es el archivo `assets/js/carrito.js`, que deberá:

Funcionalidad	Detalle
Leer el carrito	Desde <code>localStorage</code> con la clave <code>poketienda_carrito</code>
Renderizar ítems	Poblar <code>#carrito-items</code> desde el arreglo persistido
Actualizar contador	Refrescar <code>#carrito-contador</code> y <code>#carrito-cantidad-total</code>

Calcular subtotal	Sumar <code>precio × cantidad</code> por ítem
Aplicar cupón	Validar <code>PEARL10</code> (10 % descuento), actualizar <code>#carrito-descuento</code>
Gestionar cantidades	Escuchar <code>data-accion="sumar restar"</code> con event delegation
Eliminar ítems	Escuchar <code>data-accion="eliminar"</code> y re-renderizar
Vaciar carrito	Botón <code>#btn-vaciar-carrito</code> con confirmación
Estado vacío	Alternar visibilidad de <code>#carrito-items</code> y <code>#carrito-vacio</code>
Persistir cambios	Escribir en <code>localStorage</code> tras cada mutación
Poblar sugeridos	Cargar <code>#lista-sugeridos</code> con productos no presentes en el carrito
Conectar botón Pagar	<code>#btn-pagar</code> → flujo de checkout

6.2 Validación previa a la presentación

Antes de la entrega final se debe verificar:

1. **Consola del navegador sin errores** (ni 404, ni warnings de JS).
2. **Contraste AA** en todos los pares texto/fondo del carrito.
3. **Navegación por teclado** funcional en todos los controles (tabs, +, -, quitar, pagar).
4. **Persistencia** entre recargas de página (`localStorage`).
5. **Responsividad** en los 3 breakpoints (móvil, tablet, escritorio).

7. Estado del módulo público

#	Archivo	Estado	Observaciones
1	<code>index.html</code>	✓ Completo	—
2	<code>productos.html</code>	✓ Completo	—
3	<code>detalle-producto.html</code>	✓ Completo	—
4	<code>nosotros.html</code>	✓ Completo	—

5	<code>blogs.html</code>	✓ Completo	—
6	<code>detalle-blog-1.html</code>	✓ Completo	—
7	<code>detalle-blog-2.html</code>	✓ Completo	—
8	<code>contacto.html</code>	✓ Completo	—
9	<code>registro.html</code>	✓ Completo	—
10	<code>login.html</code>	✓ Completo	—
11	<code>carrito.html</code>	✓ Completo (revisado)	Contenido enriquecido, coherente con CSS dev

Cobertura HTML del módulo público: 11 de 11 archivos (100 %).

8. Conclusión

La intervención sobre `carrito.html` cumplió con los objetivos solicitados por el cliente:

1. **Preservó** íntegramente el trabajo del desarrollador CSS externo, manteniendo clases, estructura y IDs.
2. **Enriqueció** el contenido de cada ítem con información real, representativa y trazable: imagen, categoría, nombre, descripción, SKU, estado de stock, precio unitario y subtotal.
3. **Mantuvo la coherencia** con la identidad visual v2.0, con los demás archivos HTML del proyecto y con las reglas del ERS.
4. **Preparó la estructura** para la capa JavaScript, dejando los atributos `data-*` y los IDs necesarios para que `carrito.js` pueda implementarse sin retoques adicionales en el HTML.
5. **Documentó** una advertencia técnica sobre rutas de CSS que debe verificarse antes de la presentación.

El módulo público queda así con sus **11 archivos HTML completos y consistentes**, listo para la incorporación de la capa de comportamiento y para la presentación de la Evaluación Parcial 1.

Elaborado por: Desarrollador Fullstack contratado

Revisado por: Equipo de desarrollo DSY1104

Estado del documento: Aprobado para archivo

Próxima revisión: Al cierre de la implementación de `carrito.js`